DTOne Test Solution:

Error:

```
@VitoriaBSouza ➜/workspaces/DTOne-test (main) $ perl debug_assessment.pl
Ticket TCK-001 | customer=Acme | date=2026-01-23 | status=OK | latency_ms=120 | priority=P2 | sla_risk=no
Ticket TCK-002 | customer=Globex | date=2026-01-23 | status=OK | latency_ms=85 | priority=P3 | sla_risk=no
Ticket TCK-003 | FAILED | error=Malformed details field: connection_timeout at debug_assessment.pl line 74.
Totals: processed=3 ok=2 failed=1 avg_latency_ms=102.5
Slowest: ticket=TCK-001 region=us-east-1 status=OK latency_ms=120
One or more tickets failed (failed=1)
```

alt text

Before starting to troubleshoot, I decided to document what each part of the code was used for. This helped me analyze the code and its structure before beginning the debugging process.

This is the new error printed right after I added the notes to the code:

```
@VitoriaBSouza ➜/workspaces/DTOne-test (main) $ perl debug_assessment.pl
Ticket TCK-001 | customer=Acme | date=2026-01-23 | status=OK | latency_ms=120 | priority=P2 | sla_risk=no
Ticket TCK-002 | customer=Globex | date=2026-01-23 | status=OK | latency_ms=85 | priority=P3 | sla_risk=no
Ticket TCK-003 | FAILED | error=Malformed details field: connection_timeout at debug_assessment.pl line 83.
Totals: processed=3 ok=2 failed=1 avg_latency_ms=102.5
Slowest: ticket=TCK-001 region=us-east-1 status=OK latency_ms=120
One or more tickets failed (failed=1)
```

alt text

Solution:

Based on the last screenshot the terminal indicates and error on line 83 that has the parse_timeout_seconds function.

This function it parses the details value, removes the trailing “s” and converts the value into an integer. It will also throw an error if the value is malformed or missing.

```
77
78  # This function will parse details field and remove the "s" at the end
79  # Returns the timeout value as an integer
80  sub parse_timeout_seconds {
81      my ($details) = @_;
82      my ($key, $value) = split /=/, $details;
83      die "Malformed details field: $details" if !defined $value;
84      $value =~ s/s$//;
85      return $value + 0;
86  }
87
```

alt text

To troubleshoot, I printed $details to see the full value being passed before returning $value.

```
@VitoriaBSouza ➜/workspaces/DTOne-test (main) $ perl debug_assessment.pl
Ticket TCK-001 | customer=Acme | date=2026-01-23 | status=OK | latency_ms=120 | priority=P2 | sla_risk=no
Ticket TCK-002 | customer=Globex | date=2026-01-23 | status=OK | latency_ms=85 | priority=P3 | sla_risk=no
connection_timeoutTicket TCK-003 | FAILED | error=Malformed details field: connection_timeout at debug_assessment.pl line
84.
Totals: processed=3 ok=2 failed=1 avg_latency_ms=102.5
Slowest: ticket=TCK-001 region=us-east-1 status=OK latency_ms=120
One or more tickets failed (failed=1)
```

alt text

As shown on the terminal it failed while excuting the code on TCK-003 probably due to $details value is malformed or missing.

When printing the full composition of details ($key and $value) separately, it became clear that $value was empty.

```
@VitoriaBSouza ➜/workspaces/DTOne-test (main) $ perl debug_assessment.pl
Ticket TCK-001 | customer=Acme | date=2026-01-23 | status=OK | latency_ms=120 | priority=P2 | sla_risk=no
Ticket TCK-002 | customer=Globex | date=2026-01-23 | status=OK | latency_ms=85 | priority=P3 | sla_risk=no
Details:connection_timeout
Use of uninitialized value $value in concatenation (.) or string at debug_assessment.pl line 84.
Details after split: connection_timeout, key= connection_timeout, value=
Ticket TCK-003 | FAILED | error=Malformed details field: connection_timeout at debug_assessment.pl line 85.
Totals: processed=3 ok=2 failed=1 avg_latency_ms=102.5
Slowest: ticket=TCK-001 region=us-east-1 status=OK latency_ms=120
One or more tickets failed (failed=1)
```

alt text

According to the process_ticket function, the details value should come from the parsed log line using the parse_log_line function.

```
96  # This function will process each ticket and extract the information we need to print the output line
97  sub process_ticket {
98      my ($ticket) = @_;
99
100     # Parse log and extract status and region or default to "UNKNOWN"
101     my $log = parse_log_line($ticket->{log_line});
102     my $status = $log->{status} // 'UNKNOWN';
103     my $region = $log->{region} // 'UNKNOWN';
104
105     # Extract latency_ms and turn into a number integer.
106     # Will return an error if value is missings or not a number.
107     my $latency_ms = $log->{latency_ms};
108     die "Missing latency_ms for ticket $ticket->{id}" if !defined $latency_ms;
109     $latency_ms = int($latency_ms);
110
111     # Normalize priority and get opened_date
112     my $normalized_priority = normalize_priority($ticket->{priority});
113     my $opened_date = parse_date($ticket->{opened_at});
114
115     # If there is details fields will be executed otherwise will return an error
116     if ($status eq 'ERROR' && exists $log->{details}) {
117         my $timeout_s = parse_timeout_seconds($log->{details});
118         $ticket->{_timeout_s} = $timeout_s;
119     }
```

alt text

If we check the parse_log_line, it became clear that the data was being parsed by splitting only on the first “=”.

```
# This function will parse the log line we receive from the tickets
# It will return a reference of the hash
sub parse_log_line {
    my ($log_line) = @_;
    my %data;

    for my $pair (split /\s+/, $log_line) {
        next if $pair eq '';
        my ($key, $value) = split /=/, $pair;
        $data{$key} = $value;
    }
    print %data;
    return \%data;
}
```

alt text

By printing the $key and $value inside this function, I confirmed that the details key was returning only connection_timeout instead of the full string.

```perl
# This function will parse the log line we receive from the tickets
# It will return a reference of the hash
sub parse_log_line {
    my ($log_line) = @_;
    my %data;

    for my $pair (split /\s+/, $log_line) {
        next if $pair eq '';
        my ($key, $value) = split /=/, $pair;
        $data{$key} = $value;
        print "key: $key, value: $value\n"
    };
    return \%data;
}
```

```
@VitoriaBSouza ➜/workspaces/DTOne-test (main) $ perl debug_assessment.pl
key: status, value: OK
key: latency_ms, value: 120
key: region, value: us-east-1
Print data: %dataTicket TCK-001 | customer=Acme | date=2026-01-23 | status=OK | latency_ms=120 | priority=P2 | sla_risk=no
key: status, value: OK
key: latency_ms, value: 85
key: region, value: ap-south-1
Print data: %dataTicket TCK-002 | customer=Globex | date=2026-01-23 | status=OK | latency_ms=85 | priority=P3 | sla_risk=no
key: status, value: ERROR
key: latency_ms, value: 250
key: region, value: eu-west-1
key: details, value: connection_timeout
Print data: %dataTicket TCK-003 | FAILED | error=Malformed details field: connection_timeout at debug_assessment.pl line 85.
Totals: processed=3 ok=2 failed=1 avg_latency_ms=102.5
Slowest: ticket=TCK-001 region=us-east-1 status=OK latency_ms=120
One or more tickets failed (failed=1)
```

I understood that in order to solve the error I had to correct the the split. The correct approach based on my reasear for perl was to split the string into only two parts (key and value) and preserve everything after the first “=”.

More details here: https://perlmaven.com/perl-split

This allows the details field to correctly contain connection_timeout=30s.

```perl
sub parse_log_line {
    my ($log_line) = @_;
    my %data;

    for my $pair (split /\s+/, $log_line) {
        next if $pair eq '';

        #Split the keys and limit to 2 parts to avoid issues whith keys with more than 1 "="
        my ($key, $value) = split /=/, $pair, 2;
        $data{$key} = $value;
    }
    return \%data;
}
```

alt text

With this change, the parse_timeout_seconds function no longer throws an error.

```
@VitoriaBSouza ➜/workspaces/DTOne-test (main) $ perl debug_assessment.pl
key: status, value: OK
key: latency_ms, value: 120
key: region, value: us-east-1
Ticket TCK-001 | customer=Acme | date=2026-01-23 | status=OK | latency_ms=120 | priority=P2 | sla_risk=no
key: status, value: OK
key: latency_ms, value: 85
key: region, value: ap-south-1
Ticket TCK-002 | customer=Globex | date=2026-01-23 | status=OK | latency_ms=85 | priority=P3 | sla_risk=no
key: status, value: ERROR
key: latency_ms, value: 250
key: region, value: eu-west-1
key: details, value: connection_timeout=30s
Ticket TCK-003 | customer=Umbra | date=2026-01-23 | status=ERROR | latency_ms=250 | priority=P1 | sla_risk=yes
Totals: processed=3 ok=3 failed=0 avg_latency_ms=151.7
Slowest: ticket=TCK-003 region=eu-west-1 status=ERROR latency_ms=250
```

alt text

To finish I deleted all the prints added by me on the code to leave it clean.